DAY 8 | POST-READ

JavaScript and Async JavaScript: Recap and Beyond

What we covered today, plus ideas to stretch your understanding.

Prepared by **Ahmed** | linkedin.com/in/ahmed-al-imaad

Part A: Summary of today

Topic	What to remember
Variables	const by default, let when reassigning, avoid var.
Types	string, number, boolean, undefined, null, object, array. typeof checks a type.
Collections	Arrays with map, filter, reduce; objects with keys; destructuring and spread.
Loops	for...of over values, for...in over keys, classic for with an index.
Equality	Always === and !==.
Functions	Declarations, expressions and arrow functions. Default parameters.
Callbacks	Functions passed to other functions: forEach, sort, setTimeout.
Event loop	One thread runs your code; finished background work queues callbacks.
Promises	pending, then fulfilled (value) or rejected (error). .then / .catch / .finally.
async / await	await pauses that function only. try / catch for errors. Promise.all for parallel.
fetch and JSON	await fetch(url); check res.ok; await res.json(). JSON.parse and JSON.stringify.

Part B: Going beyond today

1. Promise.allSettled and Promise.race

allSettled never fails fast. race returns whichever finishes first, useful for timeouts.

```
const results = await Promise.allSettled([taskA(), taskB()]);
results.forEach(r => console.log(r.status));   // fulfilled or rejected
```

2. Optional chaining and nullish coalescing

```
const zip = order?.address?.zip;          // undefined instead of an error
const pageSize = settings.size ?? 10;     // 10 only when size is null or undefined
```

3. Timeouts with AbortController

A fetch can hang if the server is slow. AbortController lets you cancel it after a few seconds, so your app can show a friendly error instead of waiting forever.

4. Microtasks and macrotasks

Promise callbacks (.then) run before timer callbacks (setTimeout), even a 0 ms timer. Try it: log from a resolved promise and a 0 ms timer and see which prints first.

5. Modules

Split code into files with export and import (or module.exports and require in older Node.js code). You will use both styles on Day 9 and in React.

Part C: Check yourself

- What is the difference between const and let?
- Why should you prefer === over ==?
- What is a callback? Give two examples from today.
- Why does setTimeout(fn, 0) still run after the next console.log?
- What is the difference between awaiting two calls one after another and using Promise.all?

Part D: Practice ideas for tonight

- Rewrite one callback example from the notebook with async and await.
- Write groupBy(items, key) and group SmartDesk tickets by priority.
- Write a script that fetches the SmartDesk tickets and prints the three oldest open ones.

Coming up on Day 9: Node.js, npm and supply chain hygiene, building a CRUD API with Express, and testing APIs with Postman.

Otsukaresama deshita (お疲れ様でした): thank you for your hard work today